

```

1  """
2  Starter code for the problem "Cart-pole swing-up".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7
8  import time
9
10
11 from animations import animate_cartpole
12
13 import jax
14 import jax.numpy as jnp
15
16 import matplotlib.pyplot as plt
17
18 import numpy as np
19
20
21 from scipy.integrate import odeint
22
23
24
25 def linearize(f, s, u):
26     """Linearize the function `f(s, u)` around `(s, u)`.
27
28     Arguments
29     -----
30     f : callable
31         A nonlinear function with call signature `f(s, u)`.
32     s : numpy.ndarray
33         The state (1-D).
34     u : numpy.ndarray
35         The control input (1-D).
36
37     Returns
38     -----
39     A : numpy.ndarray
40         The Jacobian of `f` at `(s, u)`, with respect to `s`.
41     B : numpy.ndarray
42         The Jacobian of `f` at `(s, u)`, with respect to `u`.
43     """
44     # WRITE YOUR CODE BELOW #####
45     # INSTRUCTIONS: Use JAX to compute `A` and `B` in one line.
46     A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
47     #####
48     return A, B
49
50
51 def ilqr(f, s0, s_goal, N, Q, R, QN, eps=1e-3, max_iters=1000):
52     """Compute the iLQR set-point tracking solution.
53
54     Arguments
55     -----
56     f : callable

```

```

64     A function describing the discrete-time dynamics, such that
65     `s[k+1] = f(s[k], u[k])`.
66     s0 : numpy.ndarray
67         The initial state (1-D).
68     s_goal : numpy.ndarray
69         The goal state (1-D).
70     N : int
71         The time horizon of the LQR cost function.
72     Q : numpy.ndarray
73         The state cost matrix (2-D).
74     R : numpy.ndarray
75         The control cost matrix (2-D).
76     QN : numpy.ndarray
77         The terminal state cost matrix (2-D).
78     eps : float, optional
79         Termination threshold for iLQR.
80     max_iters : int, optional
81         Maximum number of iLQR iterations.
82
83     Returns
84     -----
85     s_bar : numpy.ndarray
86         A 2-D array where `s_bar[k]` is the nominal state at time step `k`,
87         for `k = 0, 1, ..., N-1`
88     u_bar : numpy.ndarray
89         A 2-D array where `u_bar[k]` is the nominal control at time step `k`,
90         for `k = 0, 1, ..., N-1`
91     Y : numpy.ndarray
92         A 3-D array where `Y[k]` is the matrix gain term of the iLQR control
93         law at time step `k`, for `k = 0, 1, ..., N-1`
94     y : numpy.ndarray
95         A 2-D array where `y[k]` is the offset term of the iLQR control law
96         at time step `k`, for `k = 0, 1, ..., N-1`
97
98     """
99     if max_iters <= 1:
100         raise ValueError("Argument `max_iters` must be at least 1.")
101     n = Q.shape[0] # state dimension
102     m = R.shape[0] # control dimension
103
104     # Initialize gains `Y` and offsets `y` for the policy
105     Y = np.zeros((N, m, n))
106     y = np.zeros((N, m))
107
108     # Initialize the nominal trajectory `(s_bar, u_bar)`, and the
109     # deviations `(ds, du)`
110     u_bar = np.zeros((N, m))
111     s_bar = np.zeros((N + 1, n))
112     s_bar[0] = s0
113     for k in range(N):
114         s_bar[k + 1] = f(s_bar[k], u_bar[k])
115     ds = np.zeros((N + 1, n))
116     du = np.zeros((N, m))
117
118     # iLQR loop

```

```

129     converged = False
130     for _ in range(max_iters):
131         # Linearize the dynamics at each step `k` of `(s_bar, u_bar)`
132         A, B = jax.vmap(linearize, in_axes=(None, 0, 0))(f, s_bar[:-1], u_bar)
133         A, B = np.array(A), np.array(B)
134
135         # PART (c) #####
136         # INSTRUCTIONS: Update `Y`, `y`, `ds`, `du`, `s_bar`, and `u_bar`.
137
138         # Backward pass: Riccati recursion for P (value Hessian), p (value gradient),
139         # gains Y[k] and offsets y[k]
140         P = QN
141         p = QN @ (s_bar[-1] - s_goal)
142         for k in range(N - 1, -1, -1):
143             inv = np.linalg.inv(R + B[k].T @ P @ B[k])
144             Y[k] = -inv @ B[k].T @ P @ A[k]
145             y[k] = -inv @ (R @ u_bar[k] + B[k].T @ p)
146             p = Q @ (s_bar[k] - s_goal) + A[k].T @ (p + P @ B[k] @ y[k])
147             P = Q + A[k].T @ P @ (A[k] + B[k] @ Y[k])
148
149         # Forward pass: apply policy on real dynamics
150         s_new = np.zeros_like(s_bar)
151         u_new = np.zeros_like(u_bar)
152         s_new[0] = s0
153         for k in range(N):
154             du[k] = Y[k] @ (s_new[k] - s_bar[k]) + y[k]
155             u_new[k] = u_bar[k] + du[k]
156             s_new[k + 1] = f(s_new[k], u_new[k])
157         s_bar = s_new
158         u_bar = u_new
159
160         #####
161
162         if np.max(np.abs(du)) < eps:
163             converged = True
164             break
165     if not converged:
166         raise RuntimeError("iLQR did not converge!")
167     return s_bar, u_bar, Y, y
168
169 def cartpole(s, u):
170     """Compute the cart-pole state derivative."""
171     mp = 2.0 # pendulum mass
172     mc = 10.0 # cart mass
173     L = 1.0 # pendulum length
174     g = 9.81 # gravitational acceleration
175
176     x, theta, dx, dtheta = s
177     sintheta, costheta = jnp.sin(theta), jnp.cos(theta)
178     h = mc + mp * (sintheta**2)
179     ds = jnp.array(
180         [
181             dx,
182

```

```

194         dθ,
195         (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
196         -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
197         / (h * L),
198     ]
199 )
200 )
201 return ds
202
203
204
205 # Define constants
206 n = 4 # state dimension
207 m = 1 # control dimension
208 Q = np.diag(np.array([10.0, 10.0, 2.0, 2.0])) # state cost matrix
209 R = 1e-2 * np.eye(m) # control cost matrix
210 QN = 1e2 * np.eye(n) # terminal state cost matrix
211 s0 = np.array([0.0, 0.0, 0.0, 0.0]) # initial state
212 s_goal = np.array([0.0, np.pi, 0.0, 0.0]) # goal state
213 T = 10.0 # simulation time
214 dt = 0.1 # sampling time
215 animate = False # flag for animation
216 closed_loop = True # flag for closed-loop control
217
218 # Initialize continuous-time and discretized dynamics
219 f = jax.jit(cartpole)
220 fd = jax.jit(lambda s, u, dt=dt: s + dt * f(s, u))
221
222 # Compute the iLQR solution with the discretized dynamics
223 print("Computing iLQR solution ... ", end="", flush=True)
224 start = time.time()
225 t = np.arange(0.0, T, dt)
226 N = t.size - 1
227 s_bar, u_bar, Y, y = ilqr(fd, s0, s_goal, N, Q, R, QN)
228 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
229
230 # Simulate on the true continuous-time system
231 print("Simulating ... ", end="", flush=True)
232 start = time.time()
233 s = np.zeros((N + 1, n))
234 u = np.zeros((N, m))
235 s[0] = s0
236 for k in range(N):
237     # PART (d) #####
238     # INSTRUCTIONS: Compute either the closed-loop or open-loop value of
239     # `u[k]`, depending on the Boolean flag `closed_loop`.
240     if closed_loop:
241         u[k] = u_bar[k] + Y[k] @ (s[k] - s_bar[k]) + y[k]
242     else: # do open-loop control
243         u[k] = u_bar[k]
244     #####
245     s[k + 1] = odeint(lambda s, t: f(s, u[k]), s[k], t[k : k + 2])[1]
246 print("done! ({:.2f} s)".format(time.time() - start), flush=True)
247
248 # Plot
249 fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))

```

```

plt.subplots_adjust(wspace=0.45)
labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
labels_u = (r"$u(t)$",)
for i in range(n):
    axes[i].plot(t, s[:, i])
    axes[i].set_xlabel(r"$t$")
    axes[i].set_ylabel(labels_s[i])
for i in range(m):
    axes[n + i].plot(t[:-1], u[:, i])
    axes[n + i].set_xlabel(r"$t$")
    axes[n + i].set_ylabel(labels_u[i])
if closed_loop:
    plt.savefig("cartpole_swingup_cl.png", bbox_inches="tight")
else:
    plt.savefig("cartpole_swingup_ol.png", bbox_inches="tight")
plt.show()

if animate:
    fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
    ani.save("cartpole_swingup.mp4", writer="ffmpeg")
    plt.show()

```